Term and closed-term Godel-number relations are primitive recursive

The Open Logic Project contributors

openlogic:incompleteness/arithmetization-syntax:local-snapshot-20261013; CC BY 4.0

Editorial note. The counted statement is the author proposition labelled prop:term-primrec (707-712); the proof span 714-766 is the author proof environment that immediately follows it.

Editorial note. The auxiliary routine flatten is used inside the proof and flagged inline by the author; its primitivity is a follow-up problem in the unit, so it is recorded here as a standard coding prerequisite rather than an external reference.

Editorial scope. One original result of the Open Logic Project unit incompleteness/arithmetization-syntax, counted once: Term and ClTerm are primitive recursive. Only the counted proposition and its own author proof are retained; the source is reproduced without mathematical changes and keeps the CC BY 4.0 licence and the author attribution.

Allowed standard prerequisites (editorial).

- Godel numbering of sequences. Finite sequences are coded by prime-power products; (y)_i is the i-th component of the code y, len(y) its length, and concatenation of coded symbol blocks is primitive recursive.. A formation sequence is coded as y = <G(s_0), ..., G(s_{k-1})>, and the proof reads off its entries and length (733-741).
- Variable and constant tests. Var(x) and Const(x), deciding whether x codes a variable or a constant symbol, are primitive recursive (the unit establishes them just before the counted proposition).. They are the first two clauses of the formation-rule condition on each entry (y)_i (738-740).
- Primitive recursive relations and bounded quantification. Relations built from primitive recursive functions, relations and bounded quantification are primitive recursive.. The proof stresses that the formation-sequence condition must be expressed with bounded quantifiers only, and the bounds come from the components of y (727-732, 744-752).
- Concatenation coding routine flatten. The routine flatten turning the code of a sequence of Godel numbers into the Godel number of the concatenated symbol string is primitive recursive.. It appears inside clause (3) building (y)_i from the function symbol and the argument terms (743-747); the author flags it inline and makes its primitivity a follow-up problem, so the counted proof uses it as a standard coding routine.

Proposition 0.1. *The relations* $\operatorname{fn} Term(x)$ *and* $\operatorname{fn} ClTerm(x)$ *which hold iff* x *is the Gödel number of a term or a closed term, respectively, are primitive recursive.*

Proof. A sequence of symbols s is a term iff there is a sequence s_0, ..., $s_{k-1} = s$ of terms which records how the term s was formed from !!constants and !!variables according to the formation rules for terms. To express that such a putative formation sequence follows the formation rules it has to be the case that, for each $i < k$, either

1. s_i is !!avariable v_j, or
2. s_i is !!aconstant c_j, or
3. s_i is built from n terms t_1, ..., t_n occurring prior to place i using an n-place !!function f^n_j.

To show that the corresponding relation on Gödel numbers is primitive recursive, we have to express this condition primitive recursively, i.e., using primitive recursive functions, relations, and bounded quantification.

Suppose y is the number that codes the sequence s_0, ..., s_{k-1}, i.e., $y = \langle \operatorname{gn} s_0, \ldots, \operatorname{gn} s_{k-1} \rangle$. It codes a formation sequence for the term with Gödel number x iff for all $i < k$:

1. $\operatorname{fn} Var((y)_i)$, or
2. $\operatorname{fn} Const((y)_i)$, or
3. there is an n and a number $z = \langle z_1, \ldots, z_n \rangle$ such that each z_l is equal to some $(y)_{i'}$ for $i' < i$ and

$$(y)_i = \operatorname{gn} f^n_j(\frown \operatorname{fn} flatten(z) \frown \operatorname{gn}),$$

and moreover $(y)_{k-1} = x$. (The function $\operatorname{fn} flatten(z)$ turns the sequence $\langle \operatorname{gn} t_1, \ldots, \operatorname{gn} t_n \rangle$ into $\operatorname{gn} t_1, \ldots, t_n$ and is primitive recursive.)

The indices j, n, the Gödel numbers z_l of the terms t_l, and the code z of the sequence $\langle z_1, \ldots, z_n \rangle$, in (3) are all less than y. We can replace k above with $\operatorname{len} y$. Hence we can express "y is the code of a formation sequence of the term with Gödel number x" in a way that shows that this relation is primitive recursive.

We now just have to convince ourselves that there is a primitive recursive bound on y. But if x is the Gödel number of a term, it must have a formation sequence with at most $\operatorname{len} x$ terms (since every term in the formation sequence of s must start at some place in s, and no two subterms can start at the same place). The Gödel number of each subterm of s is of course $\leq x$. Hence, there always is a formation sequence with code $\leq p^{k(x+1)}_{k-1}$, where $k = \operatorname{len} x$.

For $\operatorname{fn} ClTerm$, simply leave out the clause for !!variables. □

Proof. A sequence of symbols s is a term iff there is a sequence s_0, ..., $s_{k-1} = s$ of terms which records how the term s was formed from !!constants and !!variables according to the formation rules for terms. To express that such a putative formation sequence follows the formation rules it has to be the case that, for each $i < k$, either

1. s_i is !!avariable v_j, or
2. s_i is !!aconstant c_j, or
3. s_i is built from n terms t_1, ..., t_n occurring prior to place i using an n-place !!function f^n_j.

To show that the corresponding relation on Gödel numbers is primitive recursive, we have to express this condition primitive recursively, i.e., using primitive recursive functions, relations, and bounded quantification.

Suppose y is the number that codes the sequence s_0, ..., s_{k-1}, i.e., $y = \langle \operatorname{gn} s_0, \ldots, \operatorname{gn} s_{k-1} \rangle$. It codes a formation sequence for the term with Gödel number x iff for all $i < k$:

1. $\operatorname{fn} Var((y)_i)$, or
2. $\operatorname{fn} Const((y)_i)$, or
3. there is an n and a number $z = \langle z_1, \ldots, z_n \rangle$ such that each z_l is equal to some $(y)_{i'}$ for $i' < i$ and

$$(y)_i = \operatorname{gn} f^n_j(\frown \operatorname{fn} flatten(z) \frown \operatorname{gn}),$$

and moreover $(y)_{k-1} = x$. (The function $\operatorname{fn} flatten(z)$ turns the sequence $\langle \operatorname{gn} t_1, \ldots, \operatorname{gn} t_n \rangle$ into $\operatorname{gn} t_1, \ldots, t_n$ and is primitive recursive.)

The indices j, n, the Gödel numbers z_l of the terms t_l, and the code z of the sequence $\langle z_1, \ldots, z_n \rangle$, in (3) are all less than y. We can replace k above with $\operatorname{len} y$. Hence we can express "y is the code of a formation sequence of the term with Gödel number x" in a way that shows that this relation is primitive recursive.

We now just have to convince ourselves that there is a primitive recursive bound on y. But if x is the Gödel number of a term, it must have a formation sequence with at most $\operatorname{len} x$ terms (since every term in the formation sequence of s must start at some place in s, and no two subterms can start at the same place). The Gödel number of each subterm of s is of course $\leq x$. Hence, there always is a formation sequence with code $\leq p^{k(x+1)}_{k-1}$, where $k = \operatorname{len} x$.

For $\operatorname{fn} ClTerm$, simply leave out the clause for !!variables. □